

Lab 1: Implementation of Caesar Cipher

I. Introduction

- It is one of the simplest encryption technique in which each character in plain text is replaced by a character some fixed number of positions down to it.
- For example, if key is 3 then we have to replace character by another character that is 3 position down to it. Like A will be replaced by D, C will be replaced by F and so on.

II. Program Code

```
#include<stdio.h>

int main()
{
    char message[100], ch;
    int i, key;
    printf("Enter a message to encrypt: ");
    gets(message);
    printf("Enter key: ");
    scanf("%d", &key);
    for(i = 0; message[i] != '\0'; ++i){
        ch = message[i];
        if(ch >= 'a' && ch <= 'z'){
            ch = ch + key;
            if(ch > 'z'){
                ch = ch - 'z' + 'a' - 1;
            }
            message[i] = ch;
        }
        else if(ch >= 'A' && ch <= 'Z'){
            ch = ch + key;
            if(ch > 'Z'){
```

```
ch = ch - 'Z' + 'A' - 1;
}
message[i] = ch;
}
}
printf("Encrypted message: %s", message);
return 0;
}
```

III. Output

Enter a message to encrypt: abcd

Enter key: 3

Encrypted message: defg

Lab 2: Implementation of Vigenere Cipher

I. Introduction

Vigenere Cipher is kind of polyalphabetic substitution method. It is used for encryption of alphabetic text. For encryption and decryption Vigenere Cipher Table is used in which alphabets from A to Z are written in 26 rows.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
A	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
B	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
C	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
D	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
E	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
F	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
G	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
H	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
I	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
J	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
K	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
L	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
M	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
N	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M
O	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N
P	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
Q	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
R	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
S	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
T	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
U	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
V	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
W	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
X	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
Y	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
Z	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y

Message Text: THECRAZYPROGRAMMER

Key: HELLO

Here we have to obtain a new key by repeating the given key till its length become equal to original message length.

New Generated Key: HELLOHELLOHELLOHEL

For encryption take first letter of message and new key i.e. T and H. Take the alphabet in Vigenere Cipher Table where T row and H column coincides i.e. A.

Repeat the same process for all remaining alphabets in message text. Finally the encrypted message text is:

Encrypted Message: ALPNFHDJAFVKCLATIC

The algorithm can be expressed in algebraic form as given below. The cipher text can be generated by below equation.

$$E_i = (P_i + K_i) \bmod 26$$

II. Program Code

```
#include<stdio.h>
#include<string.h>

int main(){
    char msg[] = "THECRAZYPROGRAMMER";
    char key[] = "HELLO";
    int msgLen = strlen(msg), keyLen = strlen(key), i, j;

    char newKey[msgLen], encryptedMsg[msgLen], decryptedMsg[msgLen];

    //generating new key
    for(i = 0, j = 0; i < msgLen; ++i, ++j){
        if(j == keyLen)
            j = 0;

        newKey[i] = key[j];
    }

    newKey[i] = '\0';

    //encryption
    for(i = 0; i < msgLen; ++i)
        encryptedMsg[i] = ((msg[i] + newKey[i]) % 26) + 'A';

    encryptedMsg[i] = '\0';

    //decryption
    for(i = 0; i < msgLen; ++i)
        decryptedMsg[i] = (((encryptedMsg[i] - newKey[i]) + 26) % 26) + 'A';

    decryptedMsg[i] = '\0';

    printf("Original Message: %s", msg);
    printf("\nKey: %s", key);
    printf("\nNew Generated Key: %s", newKey);
    printf("\nEncrypted Message: %s", encryptedMsg);
    printf("\nDecrypted Message: %s", decryptedMsg);

    return 0;
}
```

III. Output

Original Message: THECRAZYPROGRMMER

Key: HELLO

Encrypted Message: ALPNFHDJAFVKCLATIC

Decrypted Message: THECRAZYPROGRAMMER

Lab 3: Implementation of Playfair Cipher

I. Introduction

The **Playfair cipher** was the first practical digraph substitution cipher. The scheme was invented in **1854** by **Charles Wheatstone** but was named after Lord Playfair who promoted the use of the cipher. In playfair cipher unlike [traditional cipher](#) we encrypt a pair of alphabets(digraphs) instead of a single alphabet.

II. Program Code

```
// C program to implement Playfair Cipher

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define SIZE 30

// Function to convert the string to lowercase
void toLowerCase(char plain[], int ps)
{
    int i;
    for (i = 0; i < ps; i++) {
        if (plain[i] > 64 && plain[i] < 91)
            plain[i] += 32;
    }
}

// Function to remove all spaces in a string
int removeSpaces(char* plain, int ps)
{
    int i, count = 0;
    for (i = 0; i < ps; i++)
        if (plain[i] != ' ')
            plain[count++] = plain[i];
    plain[count] = '\0';
    return count;
}

// Function to generate the 5x5 key square
void generateKeyTable(char key[], int ks, char keyT[5][5])
{
    int i, j, k, flag = 0, *dicty;

    // a 26 character hashmap
    // to store count of the alphabet
```

```

dicty = (int*)calloc(26, sizeof(int));
for (i = 0; i < ks; i++) {
    if (key[i] != 'j')
        dicty[key[i] - 97] = 2;
}

dicty['j' - 97] = 1;

i = 0;
j = 0;

for (k = 0; k < ks; k++) {
    if (dicty[key[k] - 97] == 2) {
        dicty[key[k] - 97] -= 1;
        keyT[i][j] = key[k];
        j++;
        if (j == 5) {
            i++;
            j = 0;
        }
    }
}

for (k = 0; k < 26; k++) {
    if (dicty[k] == 0) {
        keyT[i][j] = (char)(k + 97);
        j++;
        if (j == 5) {
            i++;
            j = 0;
        }
    }
}

}

// Function to search for the characters of a digraph
// in the key square and return their position
void search(char keyT[5][5], char a, char b, int arr[])
{
    int i, j;

    if (a == 'j')
        a = 'i';
    else if (b == 'j')
        b = 'i';

```

```

    for (i = 0; i < 5; i++) {
        for (j = 0; j < 5; j++) {
            if (keyT[i][j] == a) {
                arr[0] = i;
                arr[1] = j;
            }
            else if (keyT[i][j] == b) {
                arr[2] = i;
                arr[3] = j;
            }
        }
    }
}

// Function to find the modulus with 5
int mod5(int a) { return (a % 5); }

// Function to make the plain text length to be even
int prepare(char str[], int ptrs)
{
    if (ptrs % 2 != 0) {
        str[ptrs++] = 'z';
        str[ptrs] = '\0';
    }
    return ptrs;
}

// Function for performing the encryption
void encrypt(char str[], char keyT[5][5], int ps)
{
    int i, a[4];

    for (i = 0; i < ps; i += 2) {
        search(keyT, str[i], str[i + 1], a);

        if (a[0] == a[2]) {
            str[i] = keyT[a[0]][mod5(a[1] + 1)];
            str[i + 1] = keyT[a[0]][mod5(a[3] + 1)];
        }
        else if (a[1] == a[3]) {
            str[i] = keyT[mod5(a[0] + 1)][a[1]];
            str[i + 1] = keyT[mod5(a[2] + 1)][a[1]];
        }
    }
}

```

```

        else {
            str[i] = keyT[a[0]][a[3]];
            str[i + 1] = keyT[a[2]][a[1]];
        }
    }
}

// Function to encrypt using Playfair Cipher
void encryptByPlayfairCipher(char str[], char key[])
{
    char ps, ks, keyT[5][5];

    // Key
    ks = strlen(key);
    ks = removeSpaces(key, ks);
    toLowerCase(key, ks);

    // Plaintext
    ps = strlen(str);
    toLowerCase(str, ps);
    ps = removeSpaces(str, ps);

    ps = prepare(str, ps);

    generateKeyTable(key, ks, keyT);

    encrypt(str, keyT, ps);
}

// Driver code
int main()
{
    char str[SIZE], key[SIZE];

    // Key to be encrypted
    strcpy(key, "Monarchy");
    printf("Key text: %s\n", key);

    // Plaintext to be encrypted
    strcpy(str, "instruments");
    printf("Plain text: %s\n", str);

    // encrypt using Playfair Cipher
    encryptByPlayfairCipher(str, key);

    printf("Cipher text: %s\n", str);
}

```

```
    return 0;  
}
```

III. OUTPUT

Key text: Monarchy

Plain text: instruments

Cipher text: gatlmzclrqtX

Lab 4: Implementation of Rail Fence Cipher

I. Introduction

The rail fence cipher (also called a zigzag cipher) is a form of transposition cipher. It derives its name from the way in which it is encoded.

Plaintext	T	H	I	S	I	S	A	S	E	C	R	E	T	M	E	S	S	A	G	E
Rail Fence	T					A					T							G		
Encoding		H				S		S			E		M					A		E
key = 4			I		I				E		R				E			S		
				S						C						S				
Ciphertext	T	A	T	G	H	S	S	E	M	A	E	I	I	E	R	E	S	S	C	S

II. Program Code

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
void main()
{
    int i,j,k,l;
    char a[20],c[20],d[20];
    clrscr();
    printf("\n\nEnter the input string : ");
    gets(a);
    l=strlen(a);
    /*Encrypting*/
    for(i=0,j=0;i<l;i++)
    {
        if(i%2==0)
            c[j++]=a[i];
    }
    for(i=0;i<l;i++)
    {
        if(i%2==1)
            c[j++]=a[i];
    }
    c[j]='\0';
    printf("\nEncrypted text:");
    printf("%s",c);
    /*Decrypting*/
    if(l%2==0)
        k=l/2;
    else
        k=(l/2)+1;
    for(i=0,j=0;i<k;i++)
    {
```

```
d[j]=c[i];
j=j+2;
}
for(i=k,j=1;i<l;i++)
{
d[j]=c[i];
j=j+2;
}
d[l]='\0';
printf("\nDecryption Text: ");
printf("%s",d);
getch();
}
```

III. OUTPUT

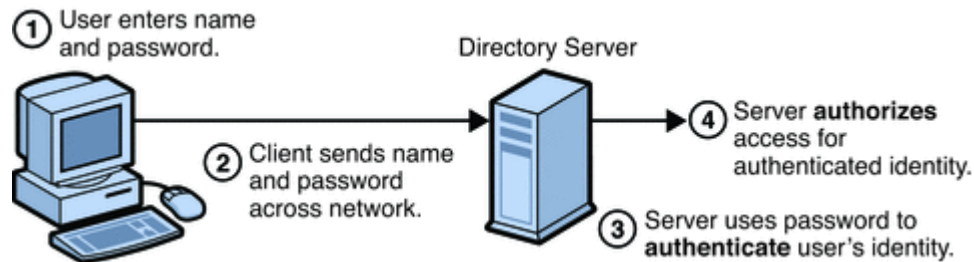
Plain Text: GEEKSFORGEEKS

Encrypted Text: GSGSEKFREKEOE

Lab 4: Implementation of Password Based Authentication System

I. Introduction

Password-Based Authentication is the process of gaining access to resources which one is entitled to with the help of a set of credentials containing username and password. This is a rampantly used method known for process simplicity and low cost.



Steps:

1. The user enters a name and password.
For the LDAP bind to Directory Server, the client application must bind with a Distinguished Name. Therefore the client application may use the name entered by the user to retrieve the DN.
2. The client sends the DN and password across the network.
3. The server determines whether the password sent from the client matches the password stored for the entry with the DN sent from the client.
If so, the server accepts the credentials as evidence authenticating the user identity.
4. The server determines whether the identified user is permitted to access the requested resource.
If so, the server allows the client to access the resource.